X-FIBF: Explainable File Integrity Behavioral Fingerprinting and Temporal API-Call Learning for Ransomware Detection

Author Name 1, Author Name 2, Author Name 3
Department of Computer Science and Engineering
Your University Name
Dhaka, Bangladesh
Email: author1@email.com, author2@email.com, author3@email.com

***Abstract*—Ransomware remains one of the most disruptive cyber threats because it combines stealthy execution, rapid file modification, encryption-oriented behavior, and financial extortion. Traditional ransomware detection methods often depend on static executable features or black-box machine-learning decisions, which limits their ability to explain temporal behavior and support safe behavioral validation. This study proposes X-FIBF, an explainable hybrid ransomware-detection framework that combines static Portable Executable (PE) analysis, dynamic API-call sequence modeling, multi-head self-attention based temporal learning, attention-based explainability, and safe File Integrity Behavioral Fingerprinting (FIBF). The proposed framework uses classical machine-learning models as static and dynamic baselines, while a Multi-Head Self-Attention Bidirectional LSTM (MHSA-BiLSTM) model is used as the primary temporal classifier for API-call sequences. The final MHSA-BiLSTM model achieved 0.9828 accuracy, 0.9521 balanced accuracy, 0.9844 malware recall, 0.9198 benign recall, 0.9922 ROC-AUC, 0.9998 PR-AUC, and 0.0156 false negative rate on the API-call test set. Attention-based analysis was used to interpret correct and incorrect predictions by highlighting influential API-call regions. In addition, a safe FIBF module was implemented using harmless dummy file-system activity to capture behavioral indicators such as burst modification, extension-changing rename activity, suspicious extension markers, and staging/deletion behavior. The FIBF pipeline was validated both on the host Windows environment and inside an isolated Ubuntu VirtualBox virtual machine, where the tuned threshold of 0.55 separated benign and suspicious activity windows in the controlled simulation. The results show that X-FIBF provides a reproducible, explainable, and ethically safe hybrid approach for ransomware behavior analysis and detection research.**

***Index Terms*—Ransomware detection, malware detection, API-call sequence, MHSA-BiLSTM, multi-head self-attention, explainable AI, file integrity monitoring, behavioral fingerprinting, FIBF.**

I. Introduction

Ransomware has become one of the most damaging cybersecurity threats because it directly compromises data availability through unauthorized file modification, encryption, locking, or deletion [1], [2]. Unlike conventional malware that may primarily emphasize persistence, surveillance, or unauthorized access, ransomware typically exposes its objective through aggressive interactions with user files and storage resources. Prior defense systems have therefore treated abnormal file access, rapid content transformation, and suspicious input/output behavior as important indicators of ransomware activity [3], [4]. These observations indicate that effective ransomware detection requires behavioral and temporal evidence in addition to static signatures or executable-level inspection.

Traditional ransomware and malware detection approaches can be broadly divided into static analysis, dynamic analysis, and behavior-based monitoring. Static analysis examines executable-level characteristics such as Portable Executable (PE) headers, imported functions, section metadata, resource information, and structural indicators. Static methods are efficient and useful for early screening, but they may be weakened by packing, obfuscation, polymorphism, and code transformation. Dynamic analysis observes runtime behavior, including API-call sequences, file-system operations, registry activity, process behavior, and network communication. Dynamic methods are more suitable for capturing behavioral signals, but they often require careful experimental design, safe execution conditions, and interpretable analysis.

Machine-learning and deep-learning techniques have been widely explored for malware and ransomware detection. Classical models such as Random Forest, XGBoost, and LightGBM can achieve strong performance on structured static or dynamic features. However, when temporal API-call sequences are flattened into fixed feature vectors, the sequential relationships between calls may be weakened. Deep sequence models, including Long Short-Term Memory (LSTM) and Bidirectional LSTM (BiLSTM), are more suitable for modeling ordered behavioral traces. Nevertheless, deep-learning models are often criticized for limited interpretability, particularly in security applications where analysts need to understand why an alert was generated.

Explainability is therefore a critical requirement in ransomware detection. A practical security system should not only output a malicious or benign prediction but should also provide supporting evidence. For temporal malware detection, such evidence may include important API-call positions, repeated suspicious token regions, or abnormal sequence patterns. For file-system behavior, useful evidence may include rapid

modification bursts, extension-changing rename operations, suspicious extension markers, staging activity, and deletion behavior. A hybrid detection system that combines prediction confidence with interpretable behavioral evidence can better support security analysts and reduce blind reliance on black-box outputs.

Another challenge in ransomware research is safe behavioral validation. Executing real ransomware is dangerous and requires strict containment, specialized malware-handling procedures, and professional sandboxing infrastructure. Many academic environments cannot safely execute ransomware samples. Therefore, safe simulation-based behavioral validation can provide a useful intermediate step. Such validation should not perform real encryption, destructive actions, or exploit behavior. Instead, it should use harmless dummy file operations to evaluate whether file-integrity behavioral features can capture suspicious ransomware-like patterns.

To address these challenges, this study proposes **X-FIBF**, an explainable hybrid framework for ransomware and malware detection. The framework integrates static PE feature-based baseline detection, dynamic API-call sequence modeling, Multi-Head Self-Attention Bidirectional LSTM (MHSA-BiLSTM) temporal classification, attention-based explainability, and safe File Integrity Behavioral Fingerprinting (FIBF). The static PE component establishes an executable-level benchmark, while the API-call sequence component captures temporal behavioral patterns. The MHSA-BiLSTM model is used as the primary temporal detector, and attention-based analysis is used to interpret the model's decisions. The FIBF module provides an additional interpretable behavioral layer by monitoring harmless file-system events and computing window-level behavioral risk scores.

The main contributions of this work are summarized as follows:

- We propose X-FIBF, a hybrid ransomware-detection framework that integrates static PE analysis, dynamic API-call sequence learning, attention-based explainability, and file-integrity behavioral fingerprinting.
- We implement and evaluate a MHSA-BiLSTM temporal detection model for API-call sequence classification, achieving strong balanced accuracy, high malware recall, high PR-AUC, and low false negative rate.
- We perform attention-based explainability analysis to inspect correct malware predictions, correct benign predictions, false positives, and false negatives by identifying influential API-call regions.
- We conduct ablation, multi-seed stability, advanced calibration, and confidence-based error analyses to evaluate component contribution, robustness, probability reliability, and prediction behavior.
- We develop a safe FIBF simulation module that captures interpretable file-system behavioral indicators such as event-rate bursts, repeated modifications, extension-changing rename operations, suspicious extension markers, and staging/deletion activity.
- We validate the FIBF pipeline both on a host Windows environment and inside an isolated Ubuntu VirtualBox virtual machine using harmless dummy file operations, without executing real ransomware or destructive payloads.

The remainder of this paper is organized as follows. Section II reviews background and related work on static ransomware detection, dynamic API-call analysis, deep temporal malware detection, explainable AI, behavioral fingerprinting, and adaptive response. Section III describes the datasets and preprocessing steps. Section IV presents the proposed X-FIBF methodology. Section V discusses the experimental design. Section VI presents the results and discussion. Section VII discusses safety, limitations, and future work. Finally, Section VIII concludes the paper.

II. Background and Related Work

A. Static Feature-Based Ransomware and Malware Detection

Static analysis is one of the earliest and most widely used approaches for malware and ransomware detection. In static analysis, executable files are inspected without execution, and features such as Portable Executable (PE) headers, section metadata, import information, resource size, debug information, linker version, and DLL characteristics are extracted. These features can be used with classical machine-learning models such as Logistic Regression, Random Forest, XGBoost, and LightGBM.

Static approaches are computationally efficient and suitable for early-stage screening because they do not require the execution of suspicious binaries. However, they also have important limitations. Malware authors often use packing, obfuscation, polymorphism, and code transformation to hide malicious structure. Therefore, static features may not fully capture runtime ransomware behavior, especially when malicious functionality is activated only under specific environmental or temporal conditions [5], [6]. In this work, static PE analysis is used as a baseline component rather than the only detection mechanism.

B. Dynamic Analysis and API-Call Sequence Detection

Dynamic analysis observes the behavior of a program during execution. Instead of relying only on executable structure, dynamic methods inspect runtime signals such as API calls, process behavior, registry activity, file-system operations, and network communication. For ransomware detection, dynamic analysis is particularly important because ransomware behavior is often expressed through ordered operations such as file discovery, file opening, writing, renaming, deletion, and encryption-oriented modification [7].

API-call sequence analysis provides a structured view of how a program interacts with operating-system resources during execution. Dynamic ransomware studies have shown that Windows API calls, registry operations, file-system activity, directory operations, and dropped-file behavior can expose characteristic runtime patterns that are difficult to observe through static inspection alone [8]. However, converting an

ordered API-call trace into a fixed aggregated feature vector may weaken dependencies between earlier and later operations. Prior malware-classification research has demonstrated that recurrent and convolutional-recurrent architectures can learn discriminative representations directly from system-call sequences [9]. Therefore, X-FIBF preserves API-call order and treats each trace as temporal behavioral evidence rather than as an unordered collection of features.

C. Deep Temporal Models for Malware Detection

Deep-learning models have been increasingly applied to malware and ransomware detection because they can learn complex nonlinear patterns from high-dimensional data. Recurrent neural networks, especially Long Short-Term Memory (LSTM) networks, are useful for sequence learning because they can capture dependencies across ordered events [10]. A Bidirectional LSTM further improves sequence modeling by learning both forward and backward temporal context [11]. This is useful when malicious behavior depends not only on previous API calls but also on later sequence patterns.

However, standard LSTM or BiLSTM models may compress sequence information into limited internal representations and may not explicitly indicate which temporal regions contributed most to the decision. To address this limitation, attention mechanisms have been introduced in sequence modeling. Self-attention allows a model to assign different importance to different positions in a sequence, while multi-head attention learns multiple attention patterns in parallel [12]. Recent ransomware detection studies have also explored attention-enhanced recurrent models for behavioral detection [13]. Motivated by this direction, this work uses a Multi-Head Self-Attention Bidirectional LSTM model as the primary temporal API-call detector.

D. Explainable AI in Malware and Ransomware Detection

Explainability is essential in cybersecurity because detection results often require analyst interpretation, incident response decisions, and risk justification. A black-box classifier that outputs only a benign or malicious label is insufficient in many practical scenarios. Security analysts need to understand why a sample was flagged, which behavioral regions were influential, and whether the alert may represent a false positive or false negative.

Several explainability techniques have been used in malware detection, including feature-importance ranking, SHAP, LIME, saliency methods, gradient-based methods, and attention visualization [14], [15]. For static PE models, SHAP can identify important executable metadata features. For temporal deep-learning models, attention visualization can highlight influential API-call positions. In this study, explainability is addressed through static feature interpretation and attention-based temporal analysis. Attention weights are examined for correctly classified malware samples, correctly classified benign samples, false positives, and false negatives.

E. Behavioral Fingerprinting and File Integrity Monitoring

Ransomware defenses frequently exploit file-system behavior because destructive encryption and mass file manipulation leave observable traces in the sequence and rate of file operations. Prior systems have shown that bursts of write activity, rapid content transformation, repeated file replacement or rename behavior, and abnormal access patterns can support early ransomware detection and recovery [3], [16], [17]. These signals are more meaningful when they are aggregated across short temporal windows instead of being interpreted as isolated events. Accordingly, the FIBF component of X-FIBF models event rate, modification count, extension-changing rename activity, suspicious extension markers, staging, and deletion as an interpretable behavioral fingerprint.

Behavioral fingerprinting summarizes these runtime activities into measurable features. Instead of depending on a single file event, behavioral fingerprinting aggregates signals across time windows. These signals may include event rate, number of modified files, number of renamed files, extension-change count, suspicious extension count, deletion count, path depth, filename entropy, and file-size statistics. Recent work has highlighted the importance of adaptive behavior fingerprinting for ransomware resilience [18]. The proposed FIBF module follows this direction by converting raw file-system events into interpretable window-level behavioral features.

F. Adaptive Response and Reinforcement Learning

Ransomware defense is not only a detection problem but also a response optimization problem. A system must decide whether to continue monitoring, raise a warning, isolate a process, trigger backup, or block an action. Recovery-oriented defenses further show that response can include preserving cryptographic material for post-attack restoration; for example, PayBreak used dynamic hooking and a key-escrow mechanism to support recovery of files encrypted by multiple ransomware families [19]. Aggressive blocking can reduce damage but may increase false disruption, while passive monitoring may reduce false alarms but allow malicious behavior to continue. Recent studies have explored reinforcement learning for adaptive ransomware response and real-time decision-making [20].

However, reinforcement-learning-based response systems require carefully designed states, actions, reward functions, and safe simulation environments. In the current work, the response layer is kept as a threshold-based and interpretable decision concept. A DQN-based adaptive response layer is considered future work rather than a completed component of the present implementation.

G. Research Gap

Based on the reviewed directions, several research gaps remain. First, many ransomware-detection studies rely primarily on either static features or dynamic features, but fewer studies combine static benchmarking, temporal API-call learning, explainability, and file-integrity behavioral fingerprinting within a unified framework. Second, deep-learning models

often achieve strong detection performance but provide limited interpretability, which is problematic in security contexts where analysts need evidence behind alerts.

Third, API-call sequence models are often evaluated using accuracy even when the dataset is highly imbalanced. This can lead to misleading conclusions because high accuracy may hide poor benign recall, high false positive rate, or weak calibration. Fourth, ransomware behavioral validation is difficult to perform safely. Executing real ransomware requires strict containment and is not suitable for many academic environments. Therefore, there is a need for controlled and harmless behavioral simulation methods that can still capture interpretable ransomware-like indicators.

Finally, many approaches do not explicitly connect neural model confidence with independent behavioral evidence. Confidence-based analysis in this work shows that some incorrect predictions can still have high confidence. This motivates a hybrid decision strategy that combines temporal model output with file-integrity behavioral risk indicators.

H. Positioning of the Proposed X-FIBF Framework

The proposed X-FIBF framework is designed to address these gaps by integrating static PE baselines, dynamic API-call sequence learning, MHSA-BiLSTM temporal modeling, attention-based explainability, advanced evaluation, and safe file-integrity behavioral fingerprinting. The framework provides a complete evaluation pipeline that compares classical baselines, BiLSTM, and MHSA-BiLSTM models. It emphasizes security-relevant metrics such as balanced accuracy, malware recall, benign recall, false positive rate, false negative rate, MCC, ROC-AUC, PR-AUC, and calibration error.

The framework also improves interpretability by combining attention-based temporal evidence with FIBF behavioral evidence. In addition, the FIBF module is validated through harmless dummy file operations in both host and Ubuntu VM environments. Therefore, X-FIBF is positioned as an explainable and ethically safe hybrid framework for ransomware behavior analysis. It does not claim universal real-world ransomware prevention; rather, it provides a reproducible foundation for combining temporal malware detection with interpretable behavioral fingerprinting.

III. Dataset and Preprocessing

This section describes the datasets, preprocessing steps, and controlled behavioral validation sources used in this study. The proposed X-FIBF framework uses three types of input evidence: static executable features, dynamic API-call sequences, and safe file-system behavioral events.

A. Static PE Feature Dataset

The first dataset consists of Windows executable-level static Portable Executable (PE) features. It contains 62,485 samples and 18 columns, including executable metadata such as machine type, debug size, debug RVA, image version, operating system version, export information, import address table information, linker version, number of sections, stack reserve size, DLL characteristics, resource size, and Bitcoin address indicators.

The original label field was represented as *Benign*, where benign files were labeled as 1 and malicious/ransomware samples were labeled as 0. For binary model training, the label was converted as follows:

$$y = 1 - \text{Benign} \tag{1}$$

Therefore, the final classification labels are defined as:

- 0: Benign
- 1: Malicious/Ransomware

The dataset contains 27,118 benign samples and 35,367 malicious/ransomware samples. No missing values or duplicate records were found. The processed dataset was divided into training, validation, and test subsets for reproducible baseline evaluation.

B. Dynamic API-Call Sequence Dataset

The second dataset consists of dynamic API-call sequences. Each sample contains a fixed-length sequence of 100 API-call tokens. The original dataset contained 43,876 rows and 102 columns, including a hash identifier, 100 API-call positions, and a malware label. After duplicate removal, 43,872 samples remained.

The final class distribution is highly imbalanced, with 1,075 benign samples and 42,797 malware samples. The vocabulary size is 307, and the sequence length is fixed at 100. The dataset was stored in NumPy format for efficient deep-learning training and evaluation.

Table I summarizes all datasets and validation sources used in this study.

C. API-Call Dataset Split

The API-call dataset was divided into training, validation, and test subsets. The split preserved the strong class imbalance observed in the original data. Table II shows the final split distribution.

Because the malware class dominates the dataset, overall accuracy alone can be misleading. Therefore, this study emphasizes balanced accuracy, malware recall, benign recall/specificity, false positive rate, false negative rate, Matthews correlation coefficient, ROC-AUC, and PR-AUC.

D. Safe FIBF Simulation Data

To validate file-integrity behavioral fingerprinting safely, a controlled FIBF simulation was created. The simulation used two isolated folders:

- *benign_area*: normal dummy file activity
- *suspicious_area*: harmless ransomware-like dummy activity

The benign simulator generated slow and normal user-like file activities such as creating, modifying, renaming, and deleting dummy files. The suspicious simulator generated harmless ransomware-like behavioral patterns, including rapid file creation, burst modification, extension-changing rename

TABLE I: Summary of datasets and validation sources used in this study.

Data Source	Samples/Events	Features/Length	Benign	Malware/Suspicious
Static PE Dataset	62,485 samples	18 PE features	27,118	35,367
API-Call Dataset	43,872 samples	100-token sequences	1,075	42,797
Host FIBF Simulation	452 events	File-system events	96 events	356 events
Ubuntu VM FIBF Simulation	614 events	File-system events	134 events	480 events

TABLE II: Train, validation, and test distribution of the API-call sequence dataset.

Split	Total Samples	Benign	Malware
Training	30,710	752	29,958
Validation	6,581	161	6,420
Test	6,581	162	6,419

operations, suspicious extension markers, marker-note creation, and temporary staging/deletion activity.

No real ransomware, encryption payload, exploit, or destructive operation was executed. All activities were performed on dummy files inside controlled project folders.

The same simulation package was executed in two environments:

- Host Windows environment
- Isolated Ubuntu VirtualBox virtual machine

The host experiment captured 452 raw file-system events, while the Ubuntu VM experiment captured 614 raw file-system events. The VM captured a larger number of modified events, which is expected because file-system event reporting can vary across operating systems, watchdog backends, and virtualization layers.

E. Preprocessing Workflow

The preprocessing workflow consisted of the following steps:

- Duplicate removal from raw datasets.
- Binary label conversion for the static PE dataset.
- Train, validation, and test splitting.
- Fixed-length API-call sequence preparation.
- Vocabulary-size extraction for token embedding.
- Conversion of API-call sequences into NumPy arrays.
- File-system event logging for FIBF simulation.
- Window-level FIBF feature extraction using 5-second behavioral windows.

For the API-call dataset, each sequence was represented as a fixed-length token vector:

$$X_i = [x_{i,1}, x_{i,2}, \ldots, x_{i,L}] \tag{2}$$

where $L = 100$ is the sequence length and each $x_{i,j}$ is an API-call token index from the vocabulary.

For the FIBF module, raw file-system events were grouped into fixed-length windows. For each window, interpretable behavioral features were computed, including total event count, event rate, modification count, creation count, deletion count, moved/renamed count, extension-change count, suspicious-extension count, file-size statistics, path depth, and filename entropy.

F. Class Imbalance Consideration

The API-call dataset is highly imbalanced, with malware samples significantly outnumbering benign samples. This imbalance can cause a classifier to achieve high accuracy while misclassifying benign samples or producing high false positive rates. Therefore, the evaluation design does not rely only on accuracy. Instead, balanced accuracy, benign recall, false positive rate, false negative rate, MCC, ROC-AUC, and PR-AUC are used to provide a more reliable assessment.

The FIBF simulation is also not treated as a large-scale ransomware benchmark. It is used as a controlled pilot validation to test whether interpretable file-system behavioral features can separate benign dummy activity from suspicious ransomware-like dummy activity under safe experimental conditions.

IV. Proposed X-FIBF Methodology

This section presents the proposed X-FIBF framework. The framework integrates static executable analysis, temporal API-call learning, attention-based explainability, and safe file-integrity behavioral fingerprinting. Fig. 1 illustrates the complete methodology flow.

A. Overview of the X-FIBF Framework

The X-FIBF framework consists of three parallel analysis branches. The first branch uses static PE features to establish baseline detection performance. The second branch uses dynamic API-call sequences to learn temporal malware behavior using deep sequence models. The third branch uses safe file-system monitoring to extract FIBF behavioral indicators from controlled dummy file activity. These three sources are then connected through an explainable decision layer.

The static PE branch provides executable-level structural evidence, while the API-call branch provides temporal runtime behavior. The FIBF branch provides interpretable file-system behavioral evidence, such as high event rate, burst modification, extension-changing rename operations, suspicious extension markers, and staging/deletion behavior. The final decision logic combines neural malware probability, attention-based temporal evidence, and file-integrity behavioral risk score.

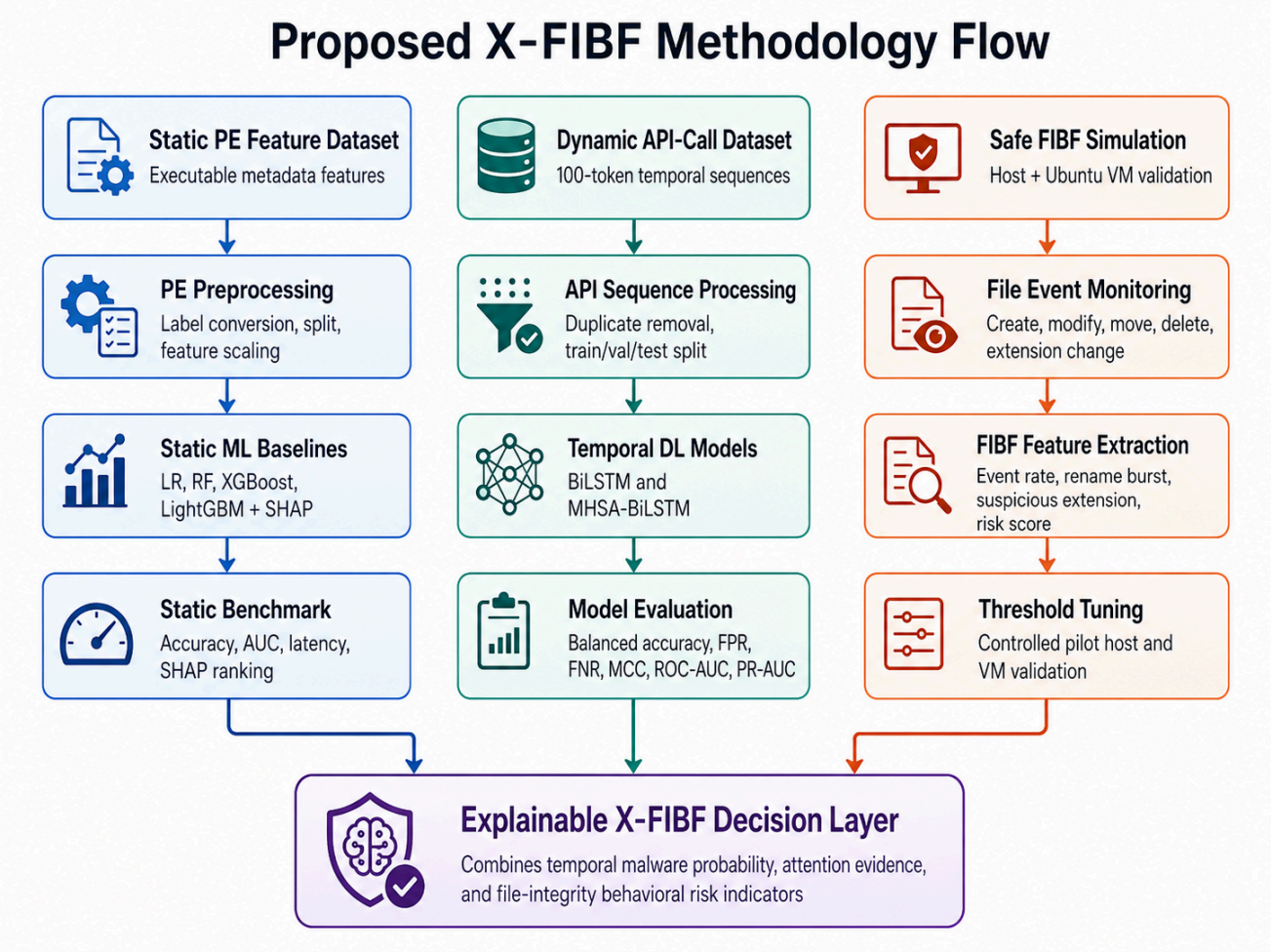

Fig. 1: Proposed X-FIBF methodology flow integrating static PE feature analysis, dynamic API-call sequence learning, attention-based explainability, and safe file-integrity behavioral fingerprinting.

B. Static PE Baseline Branch

The static branch uses PE metadata features as input to classical machine-learning models. The purpose of this branch is to establish a baseline and identify whether static executable attributes provide discriminative information. The evaluated models include Logistic Regression, Random Forest, XGBoost, and LightGBM.

The static branch is not treated as the final detection mechanism because static executable metadata cannot fully represent runtime ransomware behavior. However, it provides a useful benchmark and supports feature-level explainability through SHAP-based importance analysis.

C. Temporal API-Call Detection Branch

The dynamic branch uses API-call sequences as temporal behavioral input. Each sample is represented as a fixed-length sequence of 100 API-call tokens. These sequences are first passed through an embedding layer, where each token is mapped to a dense vector representation:

$$z_{i,t} = \text{Embedding}(x_{i,t}) \tag{3}$$

where $x_{i,t}$ denotes the API-call token at position t of sample i, and $z_{i,t}$ is its corresponding embedding vector.

The embedded sequence is then processed using a Bidirectional LSTM. The BiLSTM captures both forward and backward temporal dependencies:

$$h_t = [\overrightarrow{h_t}; \overleftarrow{h_t}] \tag{4}$$

where $\overrightarrow{h_t}$ and $\overleftarrow{h_t}$ represent forward and backward hidden states, respectively.

D. Multi-Head Self-Attention BiLSTM

The primary temporal detector in X-FIBF is the MHSA-BiLSTM model. After the BiLSTM layer, multi-head self-attention is applied to allow the model to focus on multiple important temporal regions of the API-call sequence. The scaled dot-product attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{5}$$

where Q, K, and V are query, key, and value matrices, and d_k is the key dimension. Multi-head attention applies this operation across multiple attention heads:

$$\mathrm{MHSA}(Q, K, V) = \mathrm{Concat}(head_1, \ldots, head_H)W^O \quad (6)$$

where each attention head learns a different temporal representation. A residual connection and layer normalization are then applied:

$$R = \mathrm{LayerNorm}(B + \mathrm{MHSA}(B, B, B)) \quad (7)$$

where B is the BiLSTM output and R is the attention-enhanced representation.

To summarize the sequence, mean-max pooling is applied:

$$P = [\mathrm{MeanPool}(R); \mathrm{MaxPool}(R)] \quad (8)$$

The pooled representation is passed to a fully connected classifier with sigmoid activation:

$$\hat{y} = \sigma(WP + b) \quad (9)$$

The model is optimized using binary cross-entropy loss:

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y_i}) + (1 - y_i)\log(1 - \hat{y_i})] \quad (10)$$

Algorithm 1 summarizes the training and evaluation process.

E. Attention-Based Explainability

The MHSA-BiLSTM model provides attention weights that can be used to identify influential API-call positions. In this study, attention evidence is extracted for four representative prediction groups: correctly classified malware samples, correctly classified benign samples, false positives, and false negatives. This allows the analysis to examine not only successful predictions but also failure cases.

Attention-based visualization supports model inspection by highlighting the temporal regions that received greater importance during classification. For example, a correctly classified malware sample may exhibit concentrated attention around repeated or highly discriminative API-call patterns, whereas a false positive may emphasize benign subsequences that resemble malicious behavior. However, prior research has shown that attention weights do not necessarily provide a complete or faithful causal explanation of a model's prediction [21]. Therefore, X-FIBF treats attention as supporting interpretability evidence rather than as definitive proof of causality, and analyzes it together with prediction confidence, error categories, and FIBF behavioral indicators.

Algorithm 1 MHSA-BiLSTM Training and Evaluation for API-Call Sequence Detection

Require: API-call sequences $\mathcal{X}$, labels $\mathcal{Y}$, vocabulary size V, sequence length L, training epochs E
Ensure: Trained temporal detector f_θ and evaluation metrics

Remove duplicate samples and split $\mathcal{X}, \mathcal{Y}$ into training, validation, and test sets
Convert each API-call sequence into a fixed-length token sequence of length L
Initialize embedding layer, BiLSTM layer, multi-head self-attention layer, and fully connected classifier
for epoch $= 1$ to E **do**
  for each mini-batch (X_b, Y_b) in the training set **do**
    Compute token embeddings $Z_b \leftarrow \mathrm{Embedding}(X_b)$
    Compute bidirectional representation $B_b \leftarrow \mathrm{BiLSTM}(Z_b)$
    Compute attention-enhanced output $A_b \leftarrow \mathrm{MHSA}(B_b, B_b, B_b)$
    Apply residual connection and layer normalization $R_b \leftarrow \mathrm{LayerNorm}(B_b + A_b)$
    Apply mean-max pooling $P_b \leftarrow [\mathrm{MeanPool}(R_b); \mathrm{MaxPool}(R_b)]$
    Predict malware probability $\hat{Y}_b \leftarrow \sigma(WP_b + b)$
    Compute binary cross-entropy loss
    Update model parameters θ using backpropagation
  end for
  Evaluate f_θ on the validation set
  Save the checkpoint if validation performance improves
end for
Load the best validation checkpoint
Evaluate on the test set using accuracy, balanced accuracy, recall, specificity, FPR, FNR, MCC, ROC-AUC, PR-AUC, and latency
Extract attention weights for explainability analysis
return trained model f_θ, test metrics, and attention evidence

F. FIBF Behavioral Feature Extraction

The FIBF branch monitors controlled file-system activity and converts raw events into window-level behavioral features. The monitored event types include file creation, modification, movement or renaming, deletion, and extension changes. The raw event stream is partitioned into fixed-length time windows of size Δt.

For a window w_k, the event rate is computed as:

$$r_k = \frac{n_k}{\Delta t} \quad (11)$$

where n_k is the total number of file-system events in window w_k.

The FIBF risk score is computed from normalized behavioral indicators:

$$S_k = \alpha_1\phi(r_k)+\alpha_2\phi(m_k)+\alpha_3\phi(c_k)+\alpha_4\phi(d_k)+\alpha_5\phi(g_k)+\alpha_6\phi(s_k) \tag{12}$$

where m_k is the modification count, c_k is the creation count, d_k is the deletion count, g_k is the extension-change count, s_k is the suspicious-extension count, and $\phi(\cdot)$ denotes normalization. The α values represent feature weights used to combine the behavioral indicators.

The threshold-based FIBF decision rule is:

$$\hat{y}_k = \begin{cases} 1, & \text{if } S_k \geq \tau, \\ 0, & \text{otherwise.} \end{cases} \tag{13}$$

where τ is the selected operating threshold. In the controlled host and Ubuntu VM experiments, threshold tuning selected $\tau = 0.55$.

Algorithm 2 summarizes the FIBF risk-scoring process.

G. Explainable Decision Fusion

Fig. 2 shows the explainable decision logic of X-FIBF. The decision process combines two main evidence sources. The first source is the temporal detector, which produces malware probability and attention evidence. The second source is the FIBF behavioral detector, which produces a window-level file-integrity risk score and interpretable behavioral indicators.

The final risk interpretation is divided into three conceptual states. If both temporal malware probability and FIBF risk are low, the activity is treated as benign or low risk. If the evidence is partial or uncertain, the activity is treated as suspicious or medium risk and requires closer monitoring. If both temporal malware probability and FIBF behavioral risk are high, the system assigns high risk and may trigger a response policy such as warning, isolation, backup, or blocking in future deployment.

In the current implementation, the decision fusion is used as an explainable conceptual layer rather than a fully deployed endpoint response system. Reinforcement learning-based response optimization is reserved for future work.

Algorithm 2 FIBF Window-Level Risk Scoring and Threshold Decision

Require: Raw file-system event stream $\mathcal{E}$, window size Δt, threshold τ

Ensure: Window-level FIBF predictions and behavioral evidence

Initialize empty window feature table $\mathcal{W}$
Monitor controlled directories for created, modified, moved, and deleted events
Record timestamp, event type, file path, extension, file size, and monitored area
Partition event stream $\mathcal{E}$ into fixed-length windows of size Δt
for each window w_k **do**
    Compute total event count n_k
    Compute event rate $r_k = n_k/\Delta t$
    Count created, modified, moved/renamed, and deleted events
    Count unique files touched within the window
    Count extension-changing rename events and suspicious extension markers
    Compute additional behavioral statistics such as filename entropy, path depth, and file-size variation
    Compute normalized FIBF risk score S_k
    if $S_k \geq \tau$ **then**
        Assign prediction $\hat{y}_k \leftarrow$ suspicious
    else
        Assign prediction $\hat{y}_k \leftarrow$ benign
    end if
    Store window features, risk score, prediction, and behavioral evidence in $\mathcal{W}$
end for
Tune threshold τ using balanced accuracy, FPR, FNR, and MCC
Validate selected threshold on host and Ubuntu VM controlled simulations
return window-level predictions, tuned threshold, metrics, and behavioral evidence

V. Experimental Design

This section describes the experimental environment, training configuration, evaluation metrics, threshold-tuning protocol, ablation design, stability analysis, calibration evaluation, and VM-based FIBF validation protocol.

A. Experimental Environment

The experiments were conducted across three environments: a local Windows environment, Google Colab, and an Ubuntu VirtualBox virtual machine.

The local Windows environment was used for data preprocessing, classical machine-learning baselines, local deep-learning experiments, FIBF simulation, and VM package preparation. The local Python environment was managed using Conda. The main libraries included NumPy, Pandas, Scikit-learn, XGBoost, LightGBM, PyTorch, SHAP, Matplotlib, and Watchdog.

Google Colab was used for GPU-based deep-learning experiments. The final MHSA-BiLSTM model, ablation study, multi-seed stability study, advanced evaluation, calibration analysis, and attention-based explainability were executed using a Tesla T4 GPU.

The Ubuntu VirtualBox virtual machine was used for isolated FIBF validation. The same harmless FIBF simulation package was executed inside the VM to test whether the file-integrity behavioral fingerprinting pipeline could be reproduced in a separate environment.

B. Model Training Configuration

The final MHSA-BiLSTM model was trained using the hyperparameters shown in Table III. The sequence length was fixed to 100 API-call tokens, and the vocabulary size was 307. The model used an embedding dimension of 128, BiLSTM

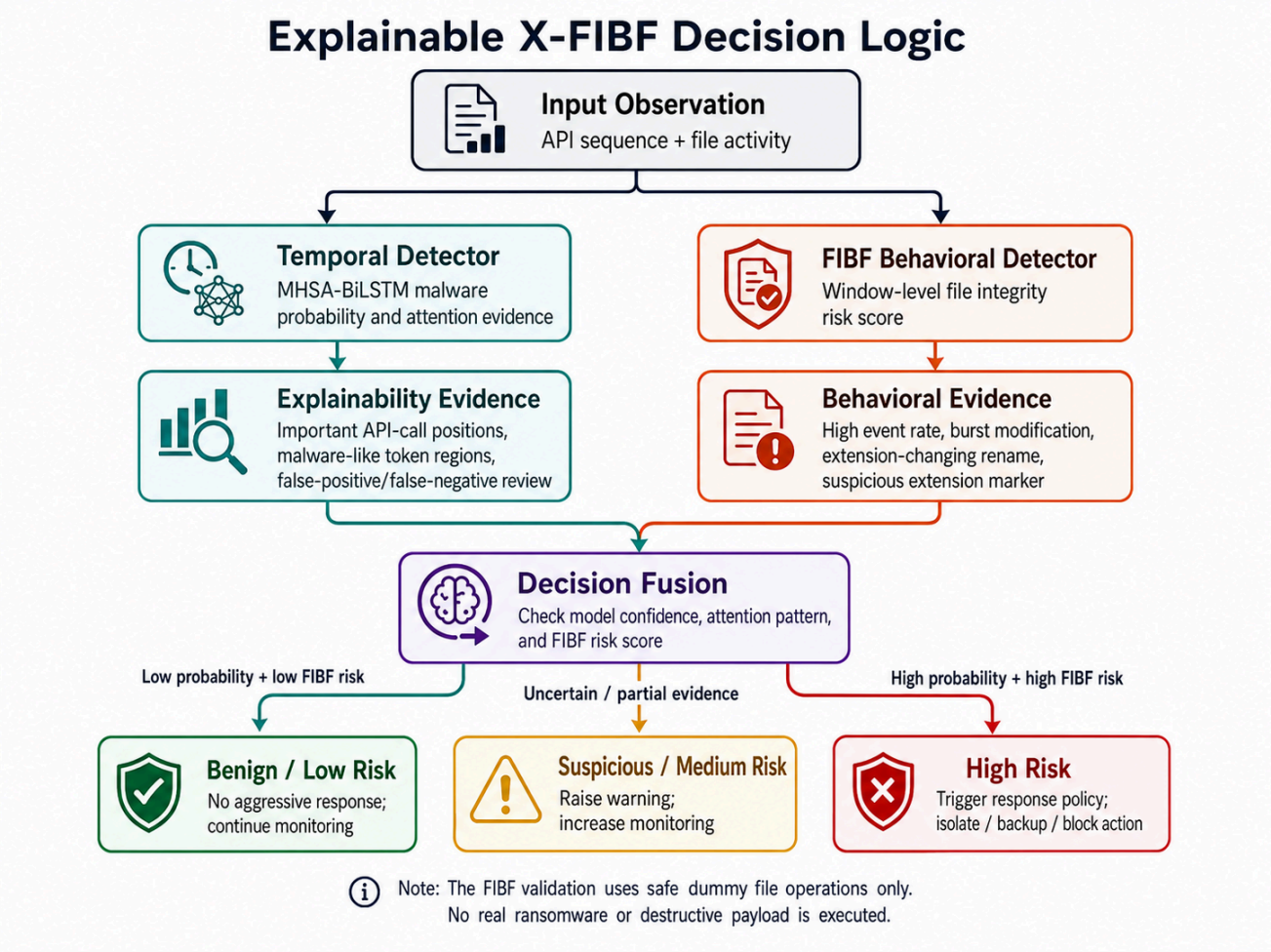

Fig. 2: Explainable X-FIBF decision logic combining temporal malware probability, attention-based evidence, and FIBF behavioral risk score for interpretable risk categorization.

TABLE III: Hyperparameter configuration of the final MHSA-BiLSTM model.

Hyperparameter	Value
Sequence length	100
Vocabulary size	307
Embedding dimension	128
BiLSTM hidden dimension	128
Attention heads	8
Dropout	0.30
Training epochs	15
Best epoch	14
Loss function	Binary cross-entropy
Primary device	Tesla T4 GPU
Framework	PyTorch

hidden dimension of 128, 8 attention heads, and dropout of 0.30. Binary cross-entropy was used as the loss function.

The best model checkpoint was selected according to validation performance and then evaluated on the held-out test set. For classical machine-learning baselines, models were trained on processed feature representations and evaluated using the same test split where applicable.

C. Evaluation Metrics

Because ransomware detection is a security-sensitive and class-imbalanced classification task, overall accuracy alone can conceal important failure patterns, particularly poor benign recognition or missed malicious samples. Precision–recall analysis is often more informative than ROC analysis when class distributions are strongly imbalanced because it focuses directly on performance for the positive class [22]. In addition, the Matthews correlation coefficient provides a balanced summary of all four confusion-matrix outcomes and remains informative even when the class sizes differ substantially [23]. Therefore, this study evaluates detection performance using accuracy, balanced accuracy, malware recall, benign recall, false positive rate, false negative rate, MCC, ROC-AUC, PR-AUC, calibration error, and inference latency.

Accuracy is defined as:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{14}$$

Balanced accuracy is defined as the average of malware recall and benign recall:

$$\text{Balanced Accuracy} = \frac{1}{2}\left(\frac{TP}{TP+FN} + \frac{TN}{TN+FP}\right) \tag{15}$$

Malware recall, also called sensitivity, is defined as:

$$\text{Recall}_{malware} = \frac{TP}{TP+FN} \tag{16}$$

Benign recall, also called specificity, is defined as:

$$\text{Recall}_{benign} = \frac{TN}{TN+FP} \tag{17}$$

The false positive rate is:

$$\text{FPR} = \frac{FP}{FP+TN} \tag{18}$$

The false negative rate is:

$$\text{FNR} = \frac{FN}{FN+TP} \tag{19}$$

The Matthews correlation coefficient is:

$$\text{MCC} = \frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}} \tag{20}$$

In addition, ROC-AUC, PR-AUC, Brier score, Expected Calibration Error (ECE), and inference latency are reported. PR-AUC is particularly important because the API-call dataset is highly imbalanced.

D. Threshold Tuning Protocol

Threshold tuning was applied to selected classical baselines and to the FIBF module. For machine-learning baselines, validation probabilities were evaluated across candidate thresholds, and the operating threshold was selected based on a balance between malware recall, benign recall, false positive rate, false negative rate, and MCC.

For the FIBF module, the window-level risk threshold was selected using a sweep over candidate thresholds. The selected FIBF threshold was:

$$\tau = 0.55 \tag{21}$$

This threshold separated benign and suspicious windows in both the host Windows simulation and Ubuntu VM validation under controlled safe conditions.

E. Ablation Study Protocol

An ablation study was conducted to evaluate the contribution of pooling and attention components. The following variants were tested:

- BiLSTM with mean pooling
- BiLSTM with mean-max pooling
- BiLSTM with single-head attention and mean-max pooling
- MHSA-BiLSTM with 4 attention heads
- MHSA-BiLSTM with 8 attention heads

The ablation study helps identify whether performance improvements are due to bidirectional sequence learning, pooling strategy, attention mechanism, or the number of attention heads.

F. Multi-Seed Stability Protocol

To evaluate stability across random initialization, a multi-seed experiment was conducted using three seeds: 42, 123, and 2026. The mean and standard deviation of major metrics were reported. This analysis was used to verify whether the model performance was stable rather than dependent on a single favorable random seed.

G. Calibration and Reliability Evaluation

Predictive accuracy does not necessarily imply that a neural classifier's confidence scores are reliable. Modern neural networks can produce poorly calibrated probabilities even when their classification performance is strong, making separate calibration analysis important for confidence-sensitive applications [24]. Therefore, the final MHSA-BiLSTM model was evaluated using the Brier score and Expected Calibration Error to measure probability quality and the agreement between predicted confidence and observed correctness. Confidence-based error analysis was also conducted across correct predictions, incorrect predictions, false positives, and false negatives.

This analysis is important because a high-confidence wrong prediction can be dangerous in a security setting. Therefore, the final X-FIBF design does not rely only on neural probability. It combines temporal model confidence with attention-based evidence and FIBF behavioral evidence.

H. Attention Explainability Protocol

Attention weights were extracted from the MHSA-BiLSTM model for representative test samples. Four categories were analyzed:

- Correctly classified malware samples
- Correctly classified benign samples
- False positive benign samples classified as malware
- False negative malware samples classified as benign

Both heatmap and line-plot visualizations were generated. These explanations were used to inspect which API-call positions contributed strongly to the prediction.

I. VM-Based FIBF Validation Protocol

The FIBF validation was first executed on the host Windows environment and then repeated inside an Ubuntu VirtualBox VM. The VM validation followed the same safe procedure:

- Run the FIBF file-system monitor.
- Execute harmless benign dummy file activity.
- Execute harmless suspicious dummy file activity.
- Stop the monitor and save raw file events.
- Extract 5-second window-level FIBF features.
- Tune and evaluate the risk-score threshold.
- Generate figures and validation summaries.

No real ransomware, encryption routine, exploit, or destructive payload was used. The purpose of the VM validation

was to test safe reproducibility of the FIBF feature-extraction pipeline under an isolated environment.

VI. Results and Discussion

This section presents the experimental results of the proposed X-FIBF framework. The evaluation was conducted in multiple stages to assess static executable classification, dynamic API-call sequence learning, deep temporal detection, explainability, robustness, calibration quality, and safe file-integrity behavioral validation. Since ransomware detection is a security-sensitive and class-imbalanced task, accuracy alone is not treated as the primary evaluation criterion. Instead, the analysis emphasizes balanced accuracy, malware recall, benign recall, false positive rate, false negative rate, Matthews correlation coefficient, ROC-AUC, PR-AUC, and inference latency.

A. Static PE Feature Baseline Results

The static PE feature dataset was first used to evaluate classical machine-learning models. The purpose of this stage was to establish a feature-based baseline using executable metadata before moving to dynamic behavioral modeling. Table IV summarizes the performance of the static PE feature-based models.

The tree-based classifiers achieved very high performance on static PE features. Random Forest, XGBoost, and LightGBM all achieved accuracy above 99%, indicating that PE metadata contains strong discriminative signals for the selected dataset. Among these models, Random Forest achieved the highest accuracy of 0.9971, while XGBoost achieved the highest ROC-AUC of 0.9997.

However, static PE features represent executable-level structural characteristics rather than runtime behavior. Therefore, although the static models provide a strong benchmark, they are not sufficient for modeling temporal ransomware behavior such as staged API usage, burst file modification, rename activity, or pre-encryption behavioral patterns. For this reason, static PE classification is treated as a supporting baseline within the proposed X-FIBF framework.

B. Dynamic API-Call Sequence Baseline Results

The second stage evaluated classical machine-learning models on the dynamic API-call sequence dataset. The API-call dataset was highly imbalanced, containing 42,797 malware samples and only 1,075 benign samples after duplicate removal. Therefore, high accuracy could be misleading, because a model may achieve strong accuracy while still failing to correctly identify benign samples or producing high false positive rates.

Table V presents the baseline results for the API-call sequence dataset.

The results show that Random Forest achieved high accuracy and malware recall, but its benign recall was only 0.4691, producing a high false positive rate of 0.5309. This illustrates the danger of relying only on accuracy in an imbalanced malware dataset. XGBoost and LightGBM achieved better trade-offs than Random Forest, but both still produced relatively high false positive rates above 0.30 at the default threshold.

To improve the balance between malware recall and benign specificity, threshold tuning was applied to XGBoost and LightGBM. Table VI shows the tuned results.

After threshold tuning, XGBoost improved its balanced accuracy to 0.8748 and reduced the false positive rate from 0.3086 to 0.2407. However, despite this improvement, the model still did not explicitly model the temporal dependency structure of API-call sequences. This motivated the use of sequence-aware deep-learning models.

C. BiLSTM and MHSA-BiLSTM Results

To capture temporal dependencies in API-call sequences, a BiLSTM model and a Multi-Head Self-Attention BiLSTM model were implemented. The BiLSTM captures bidirectional temporal context, while MHSA-BiLSTM additionally learns attention-based temporal importance across API-call positions.

Table VII compares the tuned classical baseline with the deep temporal models.

The MHSA-BiLSTM model achieved the strongest overall performance. Compared with the tuned XGBoost baseline, the final Colab-trained MHSA-BiLSTM improved balanced accuracy from 0.8748 to 0.9521 and reduced the false positive rate from 0.2407 to 0.0802. This is a substantial improvement in benign specificity while maintaining high malware recall.

The BiLSTM model improved the balance between malware detection and benign recognition compared with classical models. However, the addition of multi-head self-attention further improved the model by allowing it to focus on multiple temporally important API-call regions. The final MHSA-BiLSTM model achieved malware recall of 0.9844 and benign recall of 0.9198, indicating that it can detect most malware samples while reducing unnecessary benign misclassification.

Fig. 3 provides a visual comparison of the final dynamic detection performance.

D. Confusion Matrix Analysis

The final MHSA-BiLSTM model produced the test-set confusion matrix shown in Table VIII.

The model correctly classified 6,319 malware samples and 149 benign samples. Only 13 benign samples were incorrectly classified as malware, resulting in a false positive rate of 0.0802. Additionally, 100 malware samples were incorrectly classified as benign, giving a false negative rate of 0.0156.

In a ransomware-detection context, false negatives are particularly critical because they represent malicious samples that bypass detection. The achieved false negative rate of 0.0156 indicates that the model missed only a small proportion of malware samples in the test set. At the same time, the benign recall of 0.9198 indicates that the model avoids excessive benign disruption, which is important for practical deployment.

TABLE IV: Static PE feature-based baseline results.

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC	FPR	FNR	Latency/sample
Logistic Regression	0.8954	0.9200	0.8929	0.9063	0.9402	0.1013	0.1071	0.0004 ms
Random Forest	0.9971	0.9964	0.9985	0.9975	0.9995	0.0047	0.0015	0.0125 ms
XGBoost	0.9966	0.9966	0.9974	0.9970	0.9997	0.0044	0.0026	0.0026 ms
LightGBM	0.9970	0.9968	0.9979	0.9974	0.9996	0.0042	0.0021	0.0056 ms

TABLE V: Dynamic API-call sequence baseline results using classical machine-learning models.

Model	Accuracy	Balanced Acc.	Malware Recall	Benign Recall	FPR	FNR	MCC	ROC-AUC
Logistic Regression	0.8728	0.8385	0.8746	0.8025	0.1975	0.1254	0.3005	0.9069
Random Forest	0.9847	0.7334	0.9977	0.4691	0.5309	0.0023	0.6194	0.9826
XGBoost	0.9872	0.8430	0.9947	0.6914	0.3086	0.0053	0.7218	0.9768
LightGBM	0.9883	0.8406	0.9959	0.6852	0.3148	0.0041	0.7392	0.9697

TABLE VI: Threshold-tuned dynamic API-call baseline results.

Model	Threshold	Accuracy	Balanced Acc.	Malware Recall	Benign Recall	FPR	FNR	MCC	ROC-AUC
XGBoost	0.65	0.9847	0.8748	0.9903	0.7593	0.2407	0.0097	0.7027	0.9768
LightGBM	0.60	0.9877	0.8433	0.9952	0.6914	0.3086	0.0048	0.7296	0.9697

TABLE VII: Final dynamic detection model comparison.

Model	Accuracy	Balanced Acc.	Malware Recall	Benign Recall	FPR	FNR	MCC	ROC-AUC	Latency/sample
XGBoost Tuned	0.9847	0.8748	0.9903	0.7593	0.2407	0.0097	0.7027	0.9768	–
BiLSTM	0.9690	0.8999	0.9726	0.8272	0.1728	0.0274	0.5849	0.9667	0.1373 ms
MHSA-BiLSTM Local	0.9809	0.9511	0.9824	0.9198	0.0802	0.0176	0.7150	0.9892	1.0092 ms
MHSA-BiLSTM Colab	**0.9828**	**0.9521**	0.9844	**0.9198**	**0.0802**	**0.0156**	**0.7343**	**0.9922**	**0.0768 ms**

TABLE VIII: Confusion matrix of the final MHSA-BiLSTM model on the API-call test set.

Actual Class	Predicted Benign	Predicted Malware
Benign	149	13
Malware	100	6319

E. Ablation Study and Design Justification

To better understand which architectural components contributed most to the final performance, we conducted an ablation study on the API-call sequence dataset. The results are summarized in Fig. 4. The ablation variants included a BiLSTM model with mean pooling, a BiLSTM model with combined mean-max pooling, a single-head attention BiLSTM, and MHSA-BiLSTM configurations with different attention-head settings.

The ablation findings indicate that architectural complexity alone does not guarantee better security-oriented performance. The BiLSTM with mean-max pooling achieved a strong balance between malware recall and benign specificity, suggesting that combining average and maximum temporal evidence is highly effective for sequential malware modeling. Introducing a single attention head improved interpretability and temporal focus, but did not consistently outperform the strongest pooled BiLSTM baseline.

The multi-head self-attention variants revealed an important trade-off. While the MHSA-enhanced models improved temporal feature interaction and captured richer contextual dependencies, increasing the number of heads also introduced a tendency to prioritize malware-heavy patterns more aggressively. This behavior can improve raw accuracy, but may reduce benign specificity if not carefully controlled. Therefore, the final design was selected not merely based on highest accuracy, but on a more security-relevant balance among malware recall, benign recall, false positive rate, MCC, and interpretability.

These observations justify the overall X-FIBF modeling choice. Rather than relying on a purely static classifier or a generic sequential deep network, the proposed framework deliberately integrates temporal modeling and explainability so that performance gains remain meaningful in deployment-oriented cybersecurity settings.

F. Multi-Seed Stability Analysis

To verify that the observed performance was not an artifact of a single random initialization, we repeated the strongest compact sequential baseline across multiple random seeds. The stability experiment showed that the model maintained highly consistent results, with mean test accuracy of approximately 0.9829 ± 0.0048, balanced accuracy of 0.9521 ± 0.0061, and MCC of 0.7387 ± 0.0444. The benign recall (specificity) remained centered around 0.9198, while false positive and

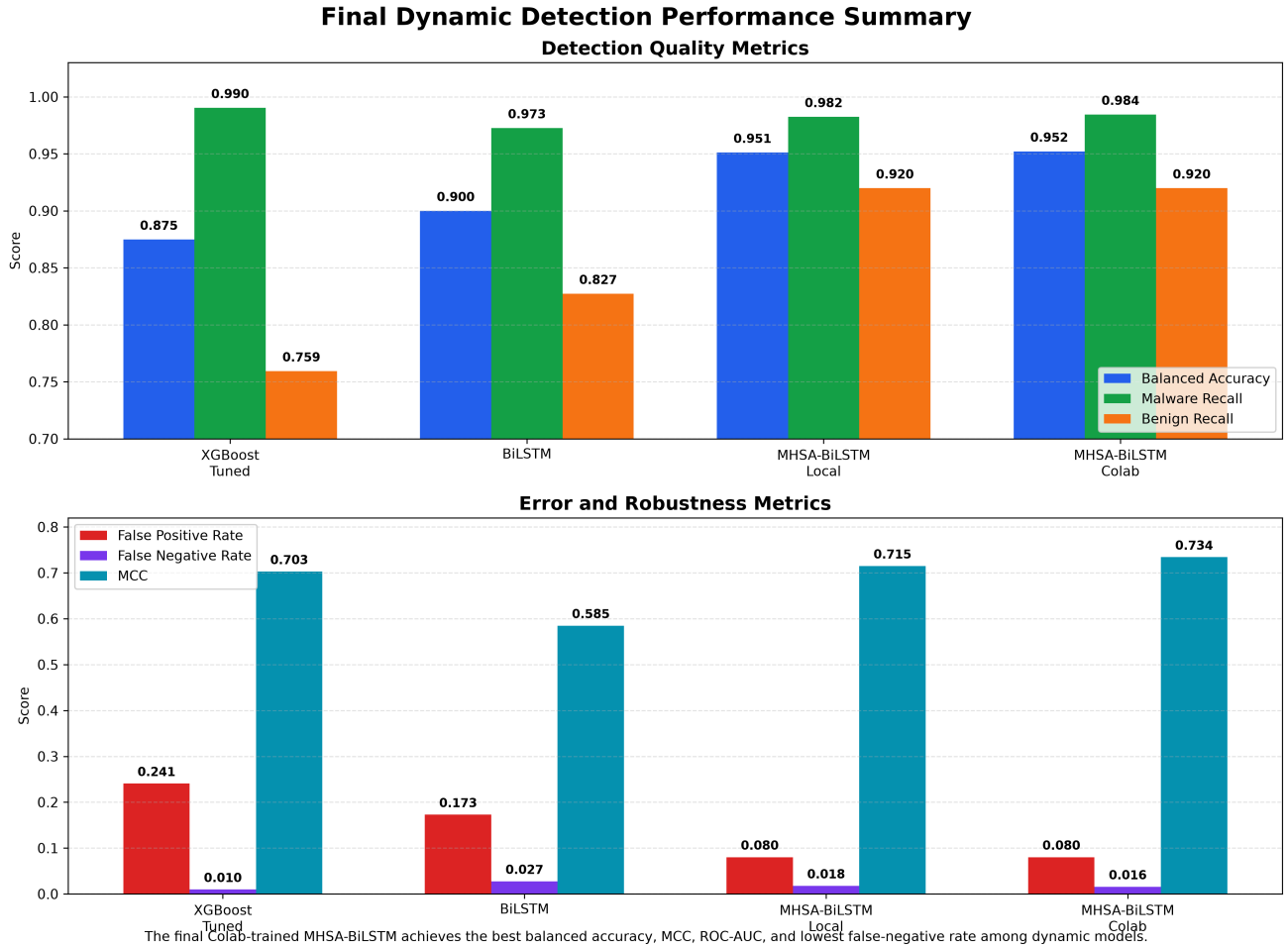

Fig. 3: Final dynamic detection performance comparison among tuned machine-learning, BiLSTM, and MHSA-BiLSTM models using balanced accuracy, recall, error rates, MCC, and related metrics.

false negative rates exhibited only limited variation across runs.

This result is important for two reasons. First, it confirms that the sequence-modeling pipeline is reproducible and not overly sensitive to stochastic training behavior. Second, it demonstrates that the proposed design decisions lead to stable discrimination between benign and malicious sequential behaviors, even under repeated training runs. Such robustness is essential for any detection framework intended for practical operational environments.

G. Calibration and Reliability Analysis

In cybersecurity, a model should not only be accurate, but should also produce confidence scores that are well calibrated. We therefore performed a reliability analysis of the final MHSA-BiLSTM model, and the results are summarized in Fig. 5.

The final model achieved a ROC-AUC of 0.9922 and a PR-AUC of 0.9998, indicating extremely strong ranking ability on the imbalanced API-call dataset. Beyond ranking quality, the model also achieved a low Brier score of 0.0151 and an expected calibration error of approximately 0.0212, suggesting that the predicted probabilities were reasonably aligned with empirical correctness. The confidence analysis further showed that correct predictions were associated with substantially higher average confidence than incorrect predictions, while false positives and false negatives tended to cluster in lower-confidence regions.

These findings strengthen the proposed framework in two ways. First, they show that the model is not only discriminative but also probabilistically reliable. Second, the confidence behavior provides a practical basis for downstream risk-aware decision strategies, such as alert prioritization, analyst review thresholds, and conservative deployment settings in high-sensitivity environments.

H. Attention-Based Explainability Analysis

A key goal of this study is to move beyond black-box detection and provide interpretable evidence for sequential ransomware-related decisions. For this purpose, we analyzed attention distributions from the MHSA-BiLSTM model for four representative cases: correctly detected malware, correctly

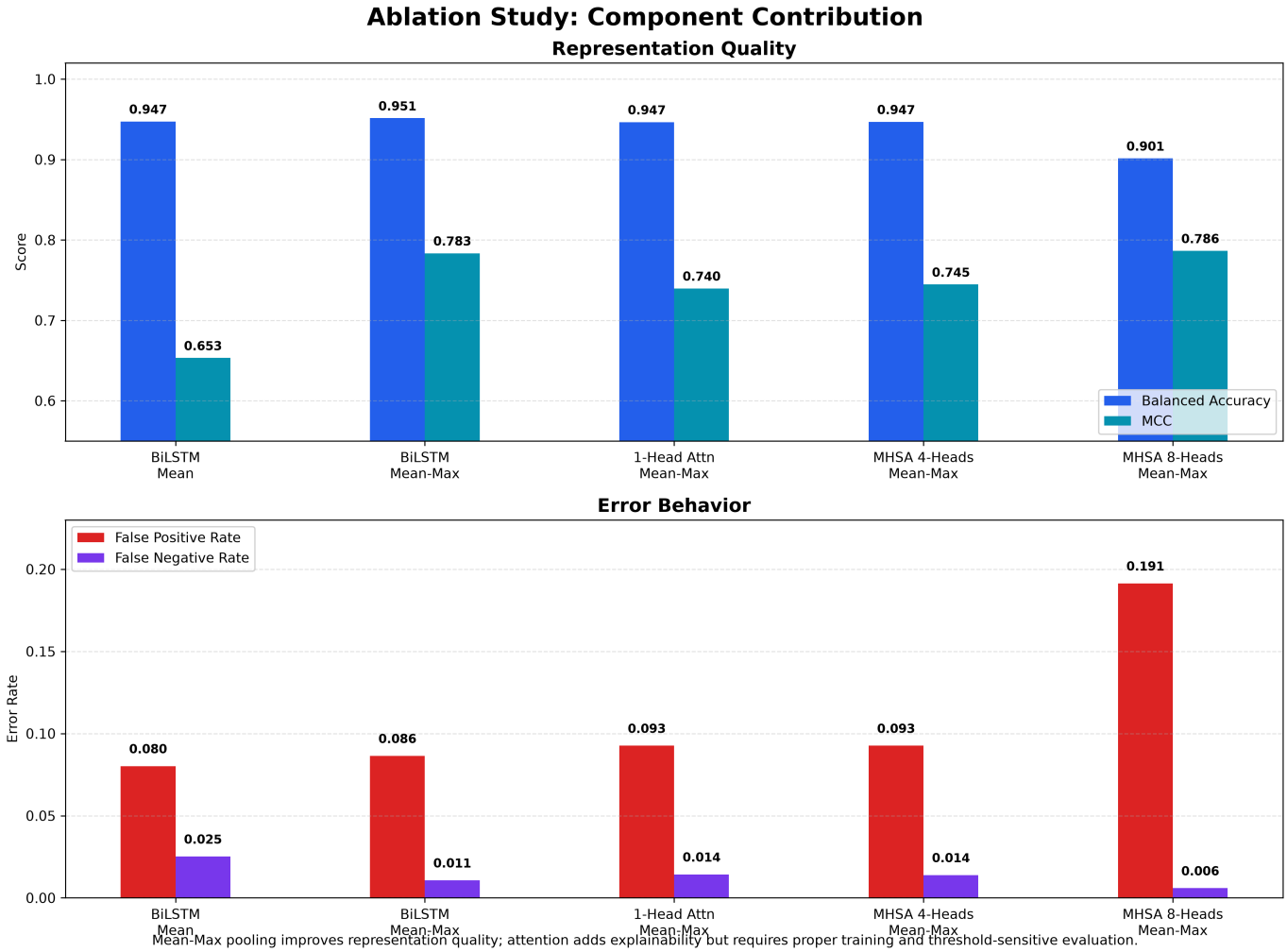

Fig. 4: Ablation analysis of sequence-model variants on the API-call dataset. The comparison highlights the influence of pooling strategy and attention configuration on balanced accuracy, benign recall, false positive rate, MCC, and inference latency.

detected benign sequences, false positives, and false negatives. The attention heatmaps are shown in Fig. 6.

The explainability analysis reveals several important behavioral patterns. In correctly detected malware samples, attention tends to concentrate on repeated or strongly discriminative API-call positions, indicating that the model learns temporally localized malicious behavior signatures rather than making diffuse or arbitrary decisions. In correctly detected benign samples, attention is distributed differently, typically emphasizing more stable and non-aggressive call patterns.

The error cases are particularly informative. False positives suggest that some benign sequences contain local temporal motifs that resemble malicious behavior, causing the model to over-emphasize suspicious subsequences. Conversely, false negatives indicate that certain malware samples may partially mimic benign temporal structure or exhibit weaker discriminative cues in the observed sequence window. This insight is valuable because it identifies where future improvements can be targeted, such as better sequence-window design, token-context enrichment, or confidence-aware post-processing.

Overall, the attention-based analysis supports the explainable nature of X-FIBF. Rather than merely reporting predictive scores, the framework provides a direct view into which temporal regions most strongly influenced the final decision. This property is especially useful in security workflows where analysts require both detection performance and interpretable justification.

I. VM-Based FIBF Validation

To complement the offline machine-learning experiments, we conducted a controlled VM-based validation of the behavioral FIBF component. This experiment simulated benign and suspicious file-activity patterns inside an isolated Ubuntu virtual machine and evaluated whether the lightweight behavioral scoring mechanism could distinguish the two conditions. The summarized results are shown in Fig. 7.

The VM experiment showed that suspicious activity generated substantially higher file-event density, higher modification and rename frequency, and stronger suspicious-extension

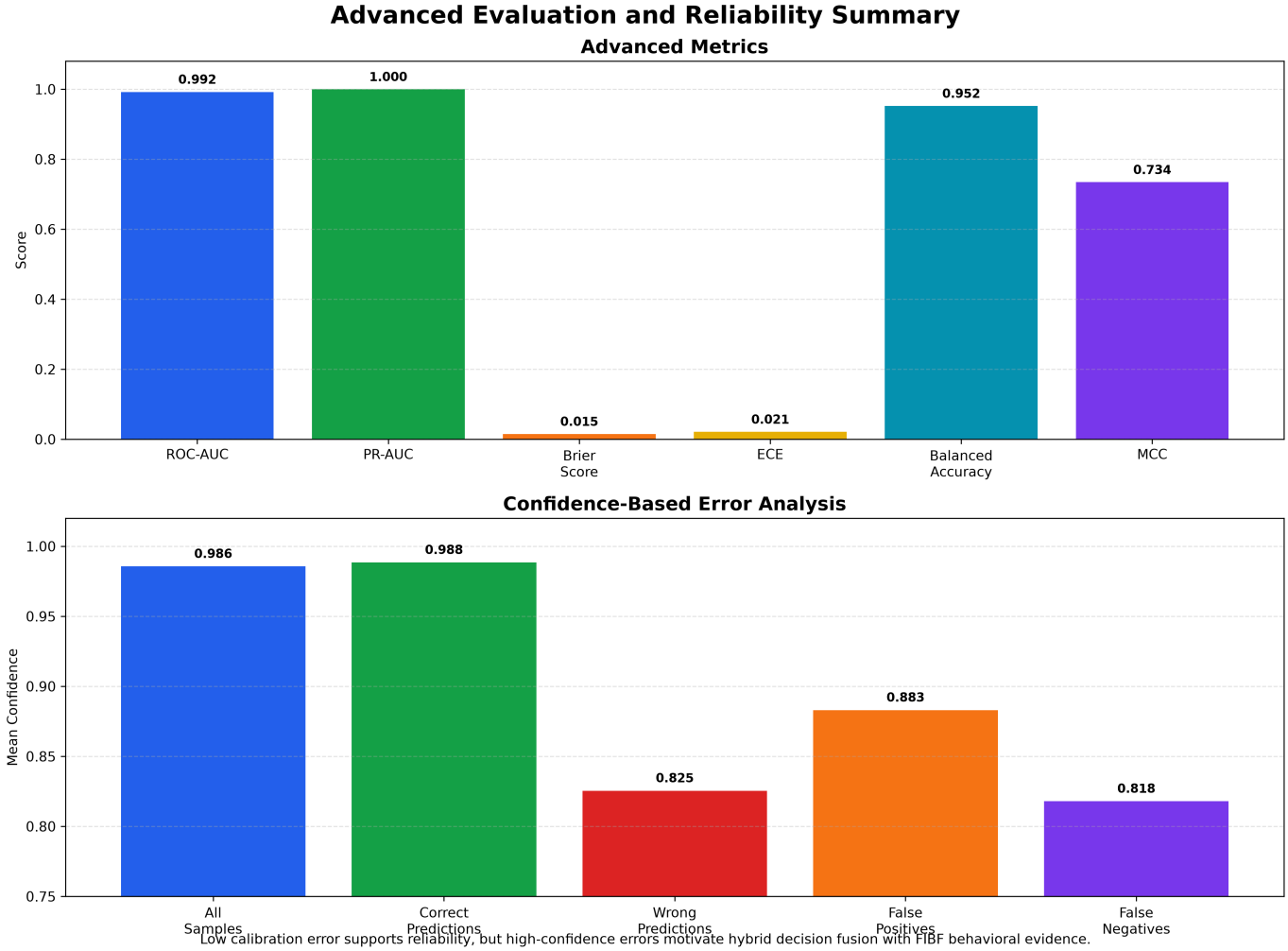

Fig. 5: Calibration and reliability analysis of the final MHSA-BiLSTM model, including ROC behavior, precision–recall behavior, calibration characteristics, and confidence distribution.

behavior than benign activity. After threshold tuning, the FIBF risk-scoring mechanism separated the small benign and suspicious windows perfectly in this controlled setting. This result does not imply that the behavioral module is already universally validated for real-world deployment; rather, it demonstrates that the proposed feature logic is operational, explainable, and capable of capturing ransomware-like file-system dynamics in a safe experimental environment.

This VM-based validation is valuable because it links the conceptual framework to an observable deployment scenario. The offline detection experiments establish predictive effectiveness on curated datasets, while the VM analysis shows that the FIBF behavioral design can be translated into a practical monitoring logic. The two together strengthen the overall contribution of the paper: X-FIBF is not only a dataset-driven detection model, but also a structured and interpretable framework that bridges static features, dynamic sequential behavior, explainability, and deployment-oriented behavioral validation.

J. Overall Discussion

Taken together, the experiments show that the proposed X-FIBF framework offers a well-rounded contribution to explainable ransomware detection. The static PE-feature experiments demonstrate that conventional machine-learning classifiers remain highly effective when strong structural metadata are available. The dynamic API-call experiments show that temporal deep learning, particularly the MHSA-BiLSTM design, can capture sequential malware behavior with strong accuracy and high ranking performance. The ablation, stability, and calibration analyses further strengthen the methodological rigor of the study, while the attention-based interpretation and VM validation improve transparency and practical relevance.

The most important contribution is therefore not only a single high-performing classifier, but an integrated research direction: explainable ransomware detection should combine predictive performance, temporal behavior modeling, decision transparency, and experimental validation in controlled environments. This perspective differentiates the present work

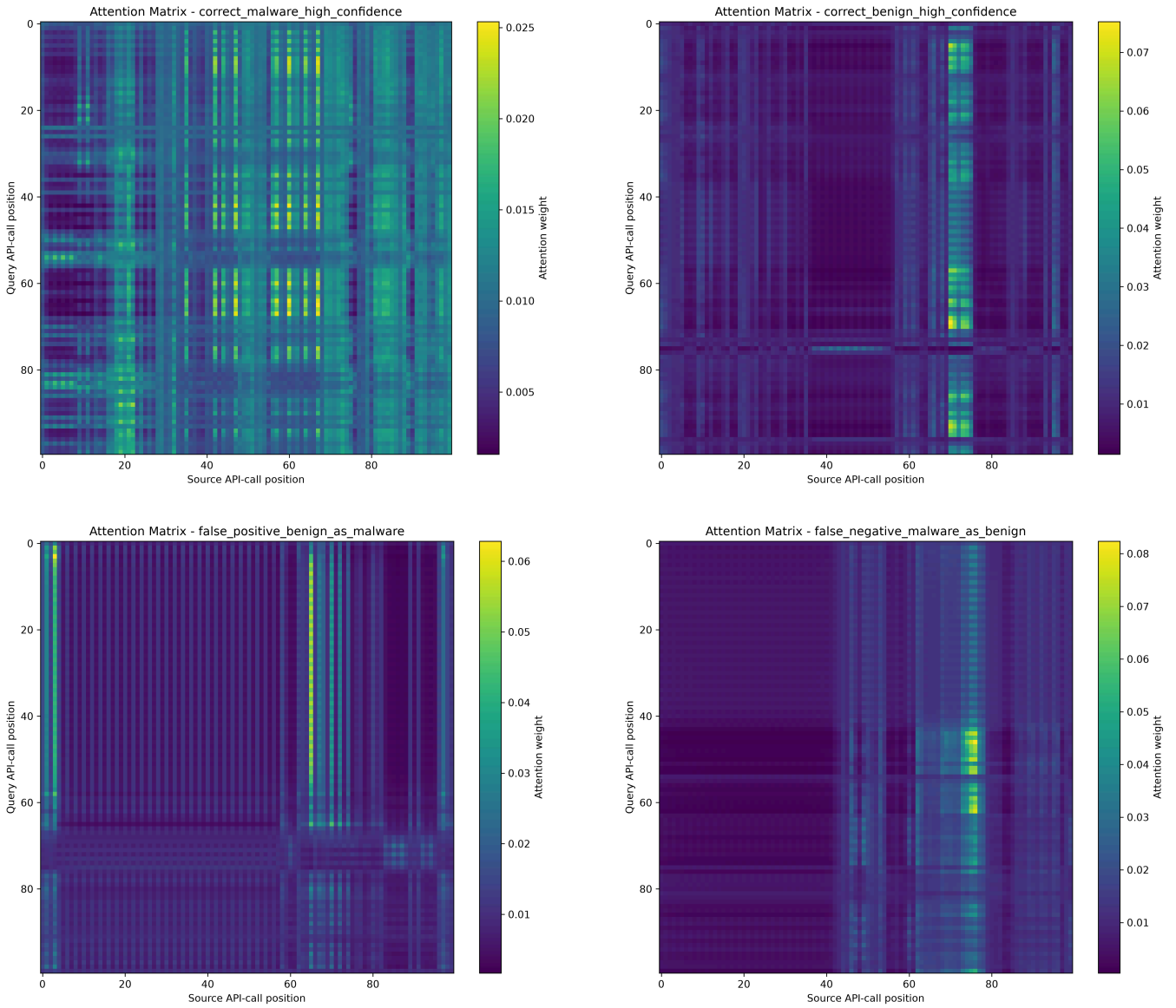

Fig. 6: Attention heatmaps from the MHSA-BiLSTM model. Top-left: correctly detected malware. Top-right: correctly detected benign sequence. Bottom-left: false positive benign sample misclassified as malware. Bottom-right: false negative malware sample misclassified as benign.

from many prior studies that report strong classification scores but provide limited insight into why the model works, how stable it is, or how it can be related to observable behavioral activity.

VII. Safety, Limitations, and Future Work

A. Safety Considerations

All behavioral validation experiments in this study were conducted under controlled and safe conditions. No real ransomware sample, real encryption payload, exploit, destructive routine, or malicious executable was executed during the FIBF simulation or VM validation. The suspicious behavioral traces were generated only through harmless dummy file operations inside isolated project folders.

The purpose of the FIBF validation was not to reproduce real ransomware destructively. Instead, the objective was to evaluate whether interpretable file-system indicators, such as rapid modification bursts, extension-changing rename operations, suspicious extension markers, and staging/deletion behavior, could be captured and transformed into window-level risk features. This design allows the behavioral component of X-FIBF to be evaluated in an ethically safe and reproducible manner.

The Ubuntu VirtualBox validation further improves the safety of the experimental setup by separating the behavioral simulation from the host environment. Although the VM experiment still used only harmless dummy activity, it demon-

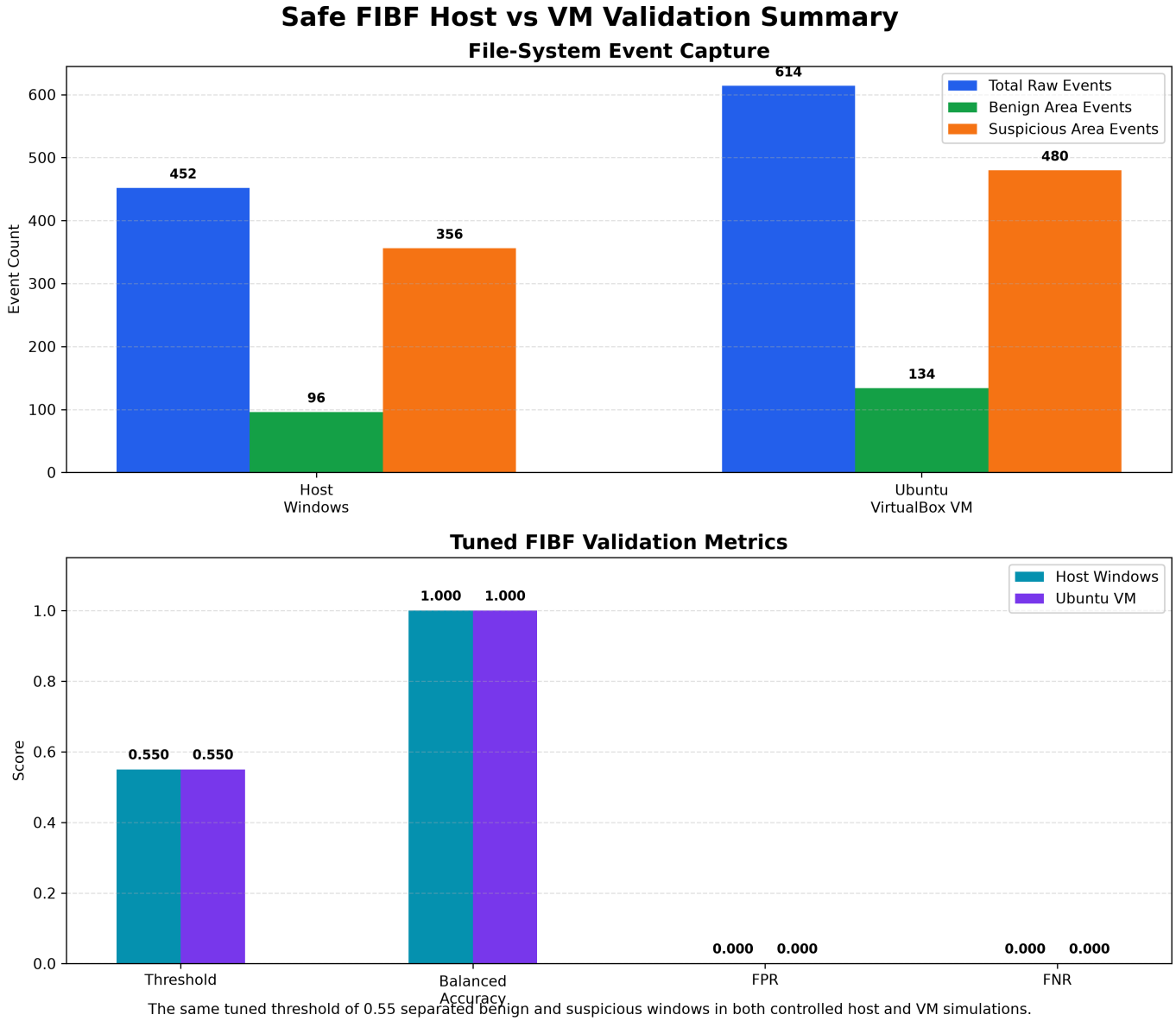

Fig. 7: Host-to-VM validation summary of the FIBF behavioral component, including event distributions, temporal risk evolution, threshold analysis, and tuned confusion behavior.

strates that the FIBF feature-extraction pipeline can operate in an isolated environment, which is an important requirement for future malware-behavior research.

B. Limitations

Despite the promising results, this study has several limitations.

First, the API-call sequence dataset is highly imbalanced, with malware samples significantly outnumbering benign samples. Although balanced accuracy, benign recall, false positive rate, false negative rate, MCC, ROC-AUC, PR-AUC, and calibration metrics were used to reduce misleading interpretation, future work should evaluate the framework on more balanced and diverse datasets.

Second, the FIBF experiment is a controlled pilot simulation rather than a large-scale real ransomware execution benchmark. The suspicious activity generator was intentionally designed to be harmless and did not perform encryption or destructive behavior. Therefore, the perfect separation observed in the controlled host and VM experiments should not be interpreted as universal ransomware detection.

Third, file-system event reporting can vary across operating systems, file-system backends, watchdog implementations, and virtualization layers. The host and VM experiments showed consistent behavioral separation, but the raw event counts differed. This indicates that environment-specific calibration may be necessary for practical deployment.

Fourth, attention-based explanations provide useful inter-

pretability evidence, but attention weights should not be interpreted as complete causal explanations. Attention visualizations can identify influential sequence positions, but additional explanation methods and analyst validation may be required for deeper forensic interpretation.

Finally, the current response layer is conceptual and threshold-based. The framework does not yet implement a complete adaptive response agent. Response optimization using reinforcement learning or cost-sensitive decision-making remains future work.

C. Future Work

Several directions can extend the proposed X-FIBF framework.

First, the framework should be evaluated on larger and more diverse ransomware datasets, including multiple ransomware families, benign enterprise workloads, and time-evolving malware variants. This would allow stronger generalization testing beyond the current API-call dataset and controlled FIBF simulation.

Second, the FIBF module can be expanded from controlled dummy activity to broader sandbox-based behavioral traces. Future work can include more realistic benign workloads, additional file-system event patterns, and environment-specific threshold calibration across Windows, Linux, and virtualized systems.

Third, the decision layer can be improved using reinforcement learning or cost-sensitive response optimization. A future DQN-based response agent may select actions such as monitor, warn, isolate, backup, or block based on temporal malware probability, FIBF risk score, alert confidence, and response cost.

Fourth, additional explainability methods can be integrated with the current attention analysis. Methods such as SHAP, LIME, Integrated Gradients, and counterfactual explanations may provide complementary evidence and help validate whether attention-highlighted regions are meaningful for analyst interpretation.

Fifth, real-time deployment considerations should be studied. Future work should evaluate streaming API-call collection, online FIBF feature extraction, low-latency inference, alert prioritization, and integration with endpoint detection and response systems.

Finally, adversarial robustness should be examined before practical deployment. Prior research has demonstrated that behavioral ransomware classifiers may be evaded by manipulating execution traces, delaying or fragmenting malicious operations, and imitating benign activity patterns [25]. Future versions of X-FIBF should therefore be evaluated against temporal slowing, altered API-call ordering, reduced event-rate bursts, suspicious-extension avoidance, and benign-behavior mimicry. Such evaluation can support the development of adaptive thresholds and more robust multi-source decision fusion.

VIII. Conclusion

This paper presented X-FIBF, an explainable hybrid framework for ransomware and malware detection that integrates static PE feature analysis, temporal API-call sequence learning, attention-based explainability, and safe file-integrity behavioral fingerprinting. The framework was designed to address three important challenges in ransomware detection research: dependence on static features alone, limited interpretability of deep-learning models, and unsafe or non-reproducible behavioral validation.

The experimental results show that the final MHSA-BiLSTM model achieved strong temporal detection performance on the API-call sequence dataset, with 0.9828 accuracy, 0.9521 balanced accuracy, 0.9844 malware recall, 0.9198 benign recall, 0.9922 ROC-AUC, 0.9998 PR-AUC, and 0.0156 false negative rate. Ablation analysis, multi-seed stability, calibration evaluation, and confidence-based error analysis further supported the robustness and reliability of the proposed temporal modeling approach.

Attention-based explainability was used to inspect correct and incorrect predictions by identifying influential API-call regions. This improved transparency and helped interpret why certain samples were classified as malware or benign. In addition, the FIBF module captured interpretable file-system behavioral indicators such as event rate, burst modification, extension-changing rename operations, suspicious extension markers, and staging/deletion behavior. The FIBF pipeline was validated both on the host Windows environment and inside an isolated Ubuntu VirtualBox virtual machine using safe dummy file operations.

Overall, X-FIBF demonstrates that combining temporal deep-learning detection, attention-based explanation, and interpretable file-integrity behavioral fingerprinting can provide a stronger and more transparent direction for ransomware detection research. The framework does not claim universal real-world ransomware prevention; rather, it provides a reproducible and ethically safe foundation for explainable ransomware behavior analysis and future adaptive response development.

References

[1] A. Kharraz, W. Robertson, D. Balzarotti, L. Bilge, and E. Kirda, "Cutting the gordian knot: A look under the hood of ransomware attacks," in *Detection of Intrusions and Malware, and Vulnerability Assessment*, ser. Lecture Notes in Computer Science, vol. 9148. Springer, 2015, pp. 3–24.

[2] A. Kharaz, S. Arshad, C. Mulliner, W. Robertson, and E. Kirda, "UNVEIL: A large-scale, automated approach to detecting ransomware," in *25th USENIX Security Symposium (USENIX Security 16)*. USENIX Association, 2016, pp. 757–772.

[3] A. Continella, A. Guagnelli, G. Zingaro, G. De Pasquale, A. Barenghi, S. Zanero, and F. Maggi, "ShieldFS: A self-healing, ransomware-aware filesystem," in *Proceedings of the 32nd Annual Conference on Computer Security Applications*. ACM, 2016, pp. 336–347.

[4] N. Scaife, H. Carter, P. Traynor, and K. R. B. Butler, "CryptoLock (and drop it): Stopping ransomware attacks on user data," in *2016 IEEE 36th International Conference on Distributed Computing Systems*. IEEE, 2016, pp. 303–312.

[5] D. W. Fernando *et al.*, "A study on the evolution of ransomware detection using machine learning and deep learning techniques," *Journal of Cybersecurity and Privacy*, 2020.

[6] S. Alzahrani *et al.*, "A survey of ransomware detection methods," *IEEE Access*, 2025.

[7] U. Urooj *et al.*, "Ransomware detection using the dynamic analysis and machine learning: A survey and research directions," *Applied Sciences*, 2022.

[8] D. Sgandurra, L. Muñoz-González, R. Mohsen, and E. C. Lupu, "Automated dynamic analysis of ransomware: Benefits, limitations and use for detection," *arXiv preprint arXiv:1609.03020*, 2016.

[9] B. Kolosnjaji, A. Zarras, G. Webster, and C. Eckert, "Deep learning for classification of malware system call sequences," in *AI 2016: Advances in Artificial Intelligence*, ser. Lecture Notes in Computer Science. Springer, 2016, pp. 137–149.

[10] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[11] M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.

[12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[13] S. A. Alzakari *et al.*, "An intelligent ransomware based cyberthreat detection model using multi head attention-based recurrent neural networks with optimization algorithm in iot environment," *Scientific Reports*, 2025.

[14] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems*, 2017, pp. 4765–4774.

[15] M. T. Ribeiro, S. Singh, and C. Guestrin, ""why should i trust you?": Explaining the predictions of any classifier," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 1135–1144.

[16] A. Kharraz and E. Kirda, "Redemption: Real-time protection against ransomware at end-hosts," in *Research in Attacks, Intrusions, and Defenses*, ser. Lecture Notes in Computer Science, vol. 10453. Springer, 2017, pp. 98–119.

[17] S. Mehnaz, A. Mudgerikar, and E. Bertino, "RWGuard: A real-time detection system against cryptographic ransomware," in *Research in Attacks, Intrusions, and Defenses*. Springer, 2018, pp. 114–136.

[18] R. Hurley *et al.*, "Real-time ransomware detection through adaptive behavior fingerprinting for improved cybersecurity resilience and defense," OSF Preprint, 2024.

[19] E. Kolodenker, W. Koch, G. Stringhini, and M. Egele, "PayBreak: Defense against cryptographic ransomware," in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*. ACM, 2017, pp. 599–611.

[20] K. Thakur *et al.*, "Real-time ransomware detection using reinforcement learning agents," *Information*, 2026.

[21] S. Jain and B. C. Wallace, "Attention is not explanation," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019, pp. 3543–3556.

[22] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[23] D. Chicco and G. Jurman, "The advantages of the matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation," *BMC Genomics*, vol. 21, no. 1, p. 6, 2020.

[24] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 70, 2017, pp. 1321–1330.

[25] F. De Gaspari, D. Hitaj, G. Pagnotta, L. De Carli, and L. V. Mancini, "Evading behavioral classifiers: A comprehensive analysis on evading ransomware detection techniques," *Neural Computing and Applications*, vol. 34, pp. 12 077–12 096, 2022.